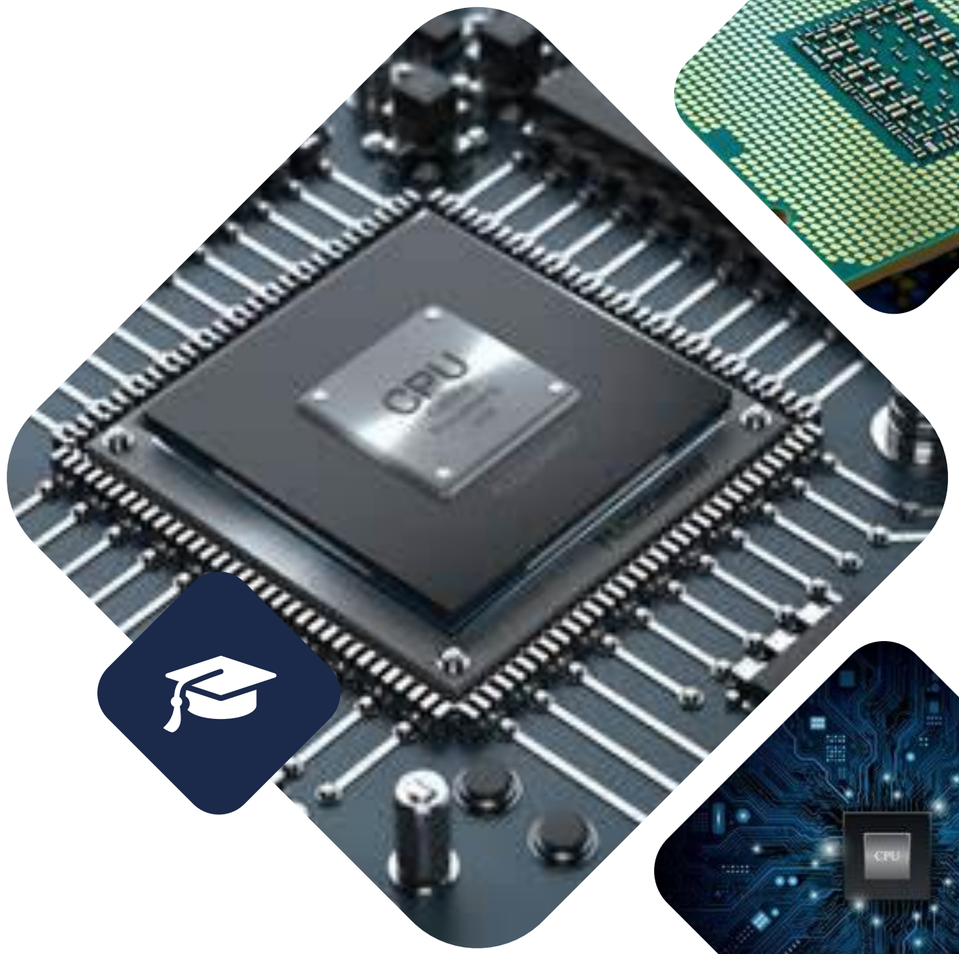
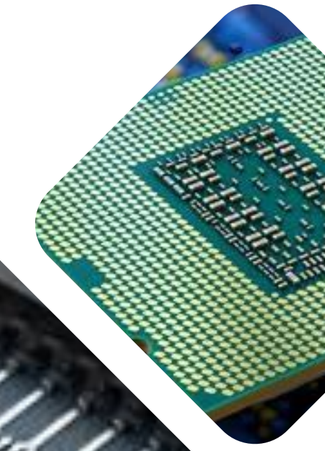
SINGLE CYCLE PROCESSER

2025





TEAM MEMBER

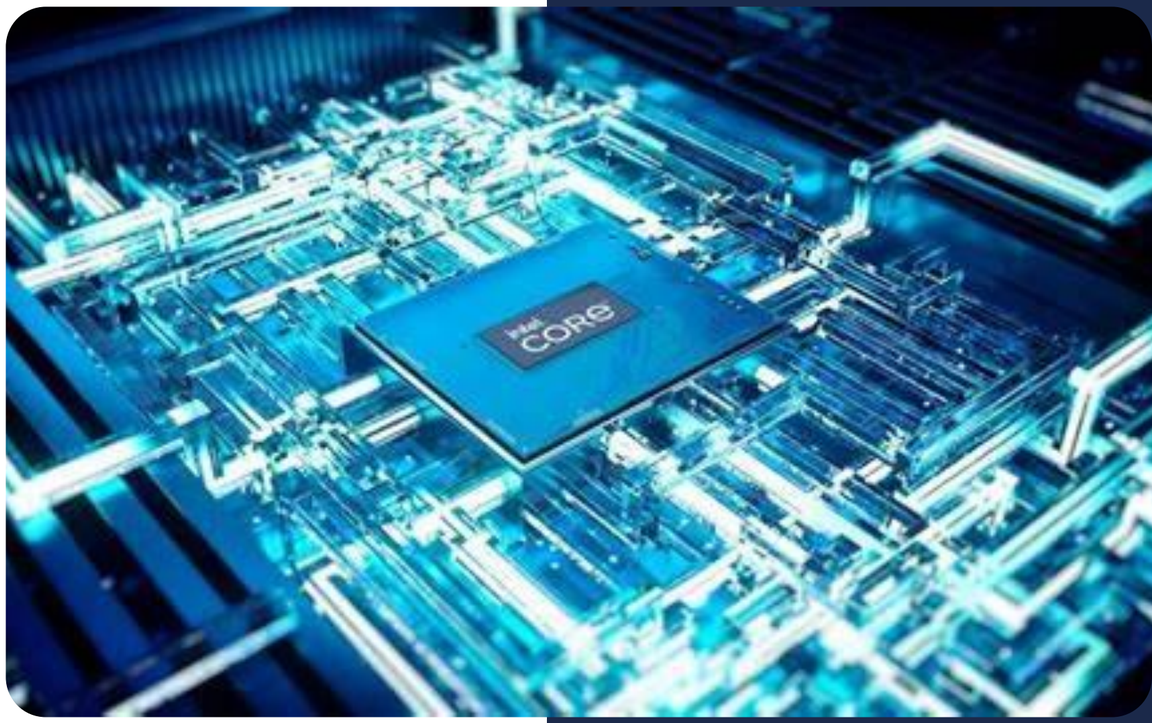
- Gehad Amr
- Eslam Ahmed
- Nada Ashraf
- Nersreen Ahmed
- Nour Abd El-Sater

Dr. LAMIA.ALREFAAI

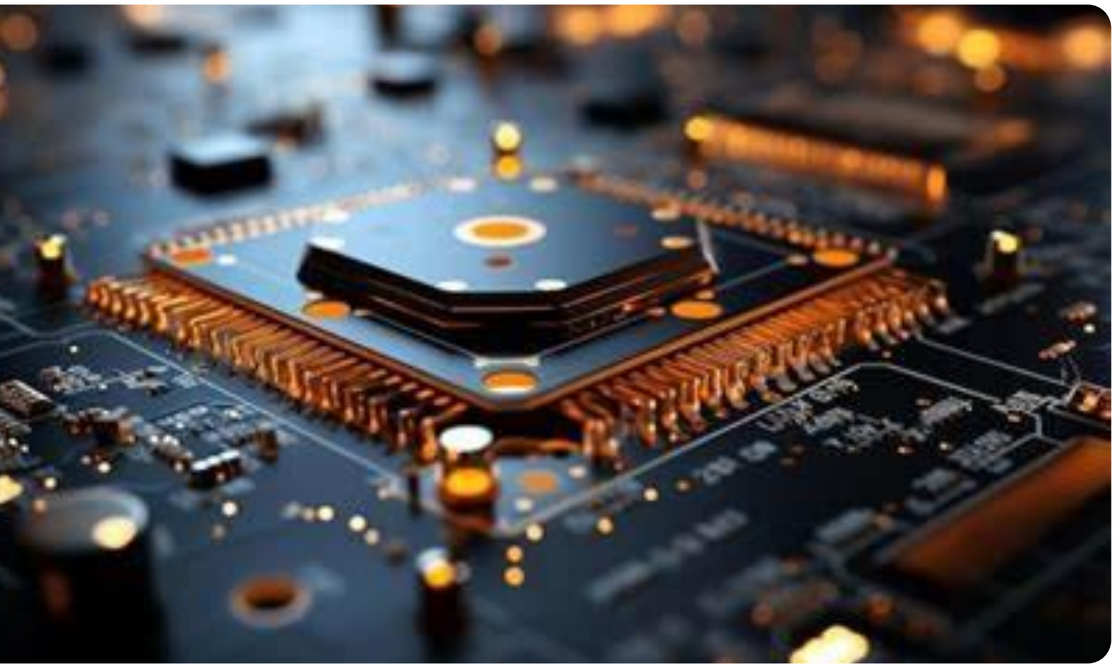
Introduction

- **Goal: To design and simulate a 32-bit processor capable of executing a minimal instruction set.**

- **Scope: The CPU supports arithmetic, conditional branching, and jump instructions.**



CPU ARCHITECTURE OVERVIEW



01 Program Counter (PC)

02 Instruction Memory

03 Control Unit

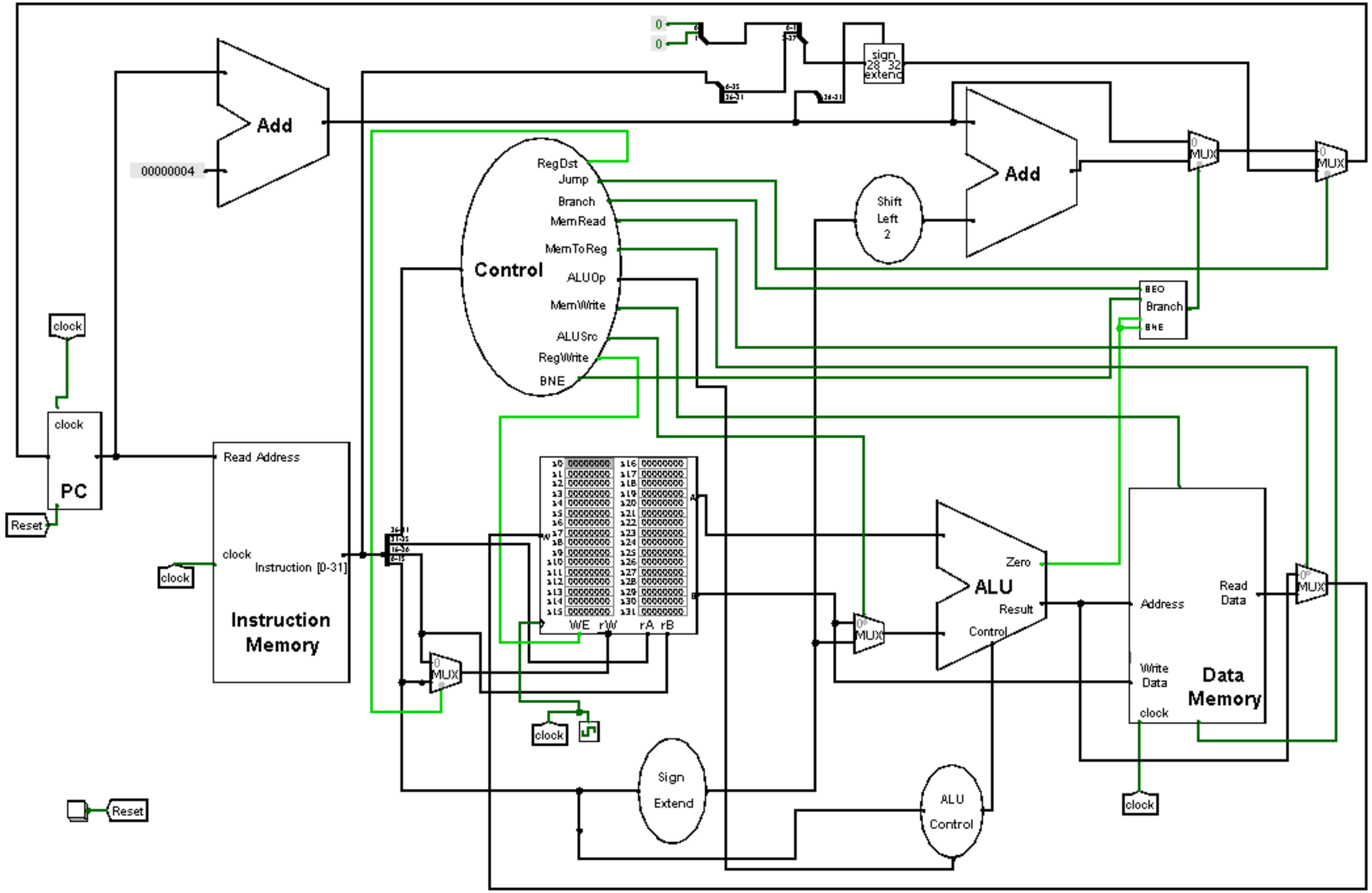
04 Register File (32 registers, each 32 bits)

05 ALU (Arithmetic Logic Unit)

06 Data Memory (if applicable)

07 Multiplexers, Sign Extenders, etc.

Single Cycle Processer



OVERVIEW

What Is a **Single-Cycle Processor**?

A single-cycle processor **executes** one **instruction** per **clock cycle**. This design combines simplicity and speed but comes at the cost of inefficiency for complex instructions. **All operations (fetch, decode, execute, memory access, write-back) happen in one clock cycle.**

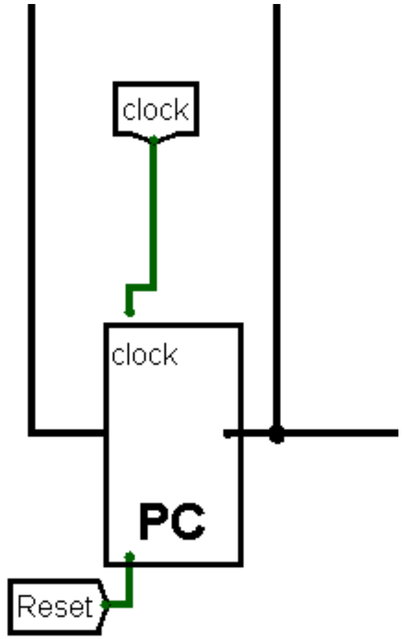
1. Program Counter (PC)

01 **Purpose**: Holds the address of the current instruction.

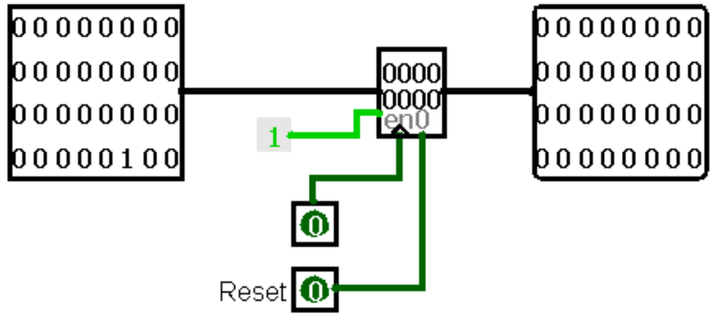
02 **How it works**:

- Initially set to 0 or a reset value.
- Increments by 4 each clock cycle unless altered by beq, bne, or jump

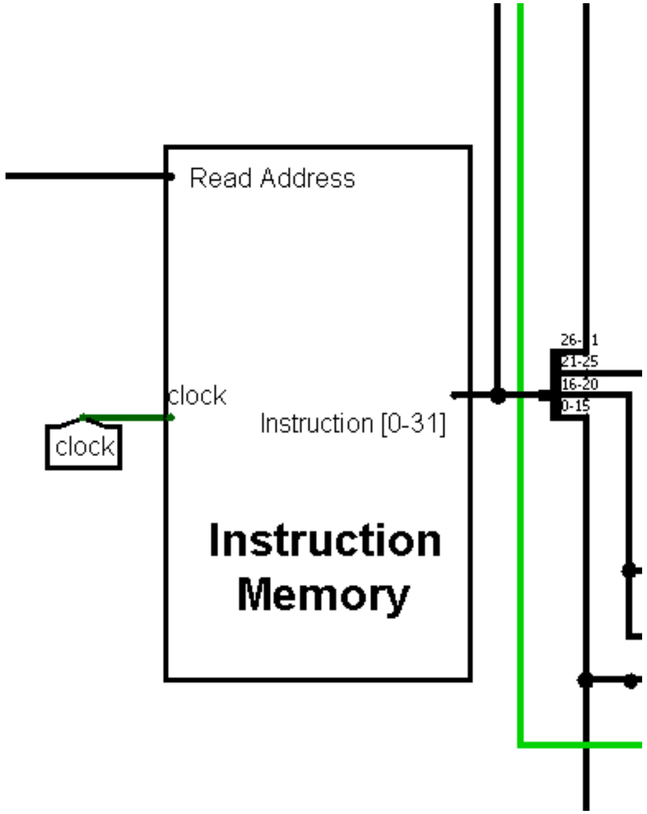
03 **Control Signal**: Clock (PC updated each cycle).



Program Counter (PC)

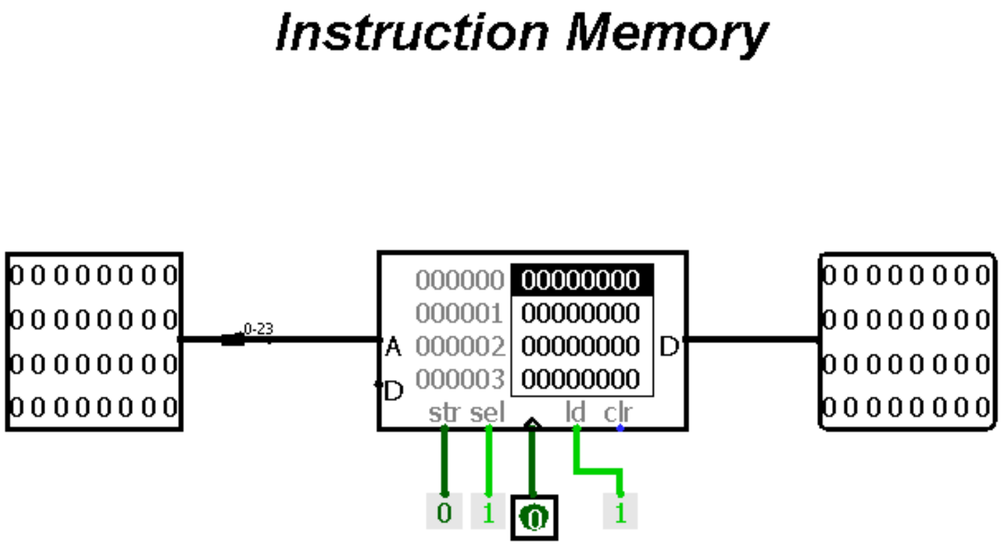


2. INSTRUCTION MEMORY



01 **Purpose**: Stores instructions for the program.

02 **Input**: PC address.



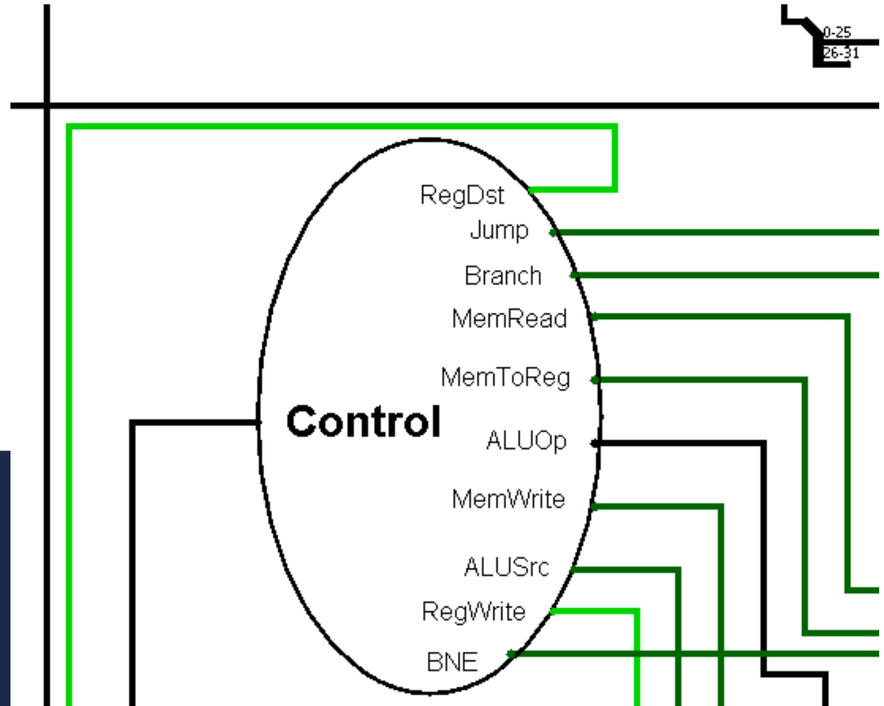
03 **Output**: 32-bit instruction.

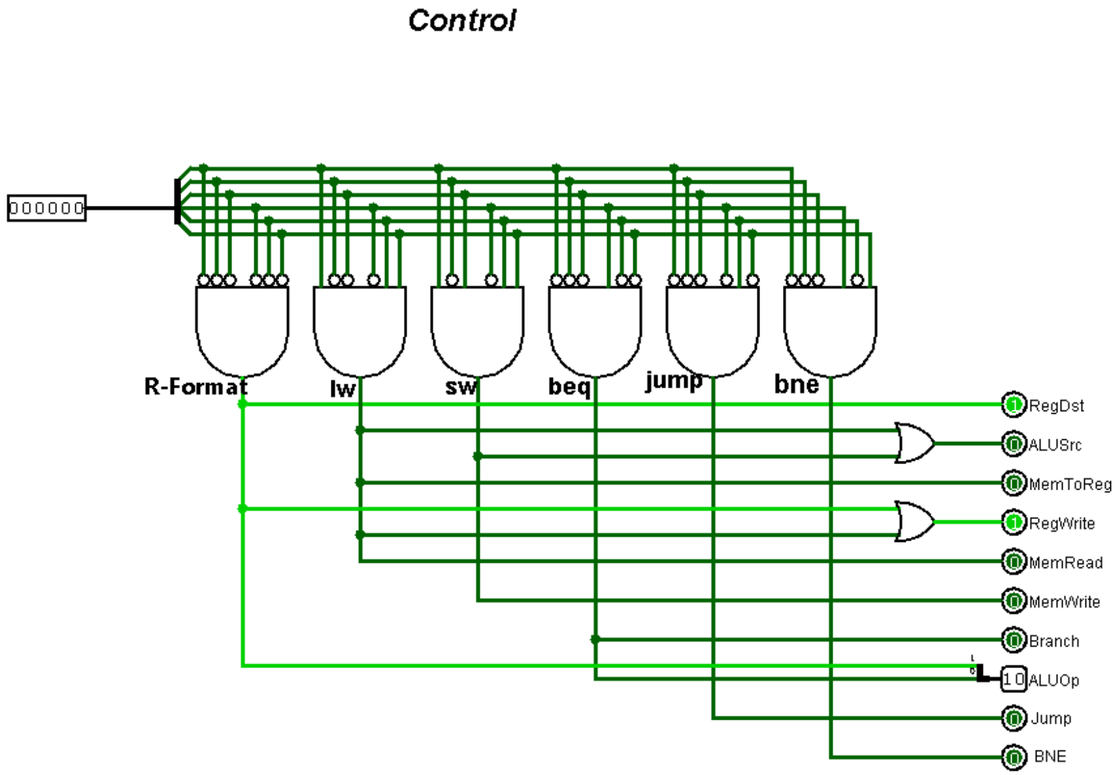
04 **Clocked?** : No — it's read-only memory, accessed combinationally.

3. CONTROL UNIT

01 **Purpose**: Generates control signals based on the instruction opcode.

02 **Input**: Instruction [31:26] (opcode).





03 **Output**: RegDst, ALUSrc, MemtoReg, RegWrite, MemRead, MemWrite, Branch, Jump, ALUOp, BNE.

04 **Determines**: The type of instruction and how other blocks behave.

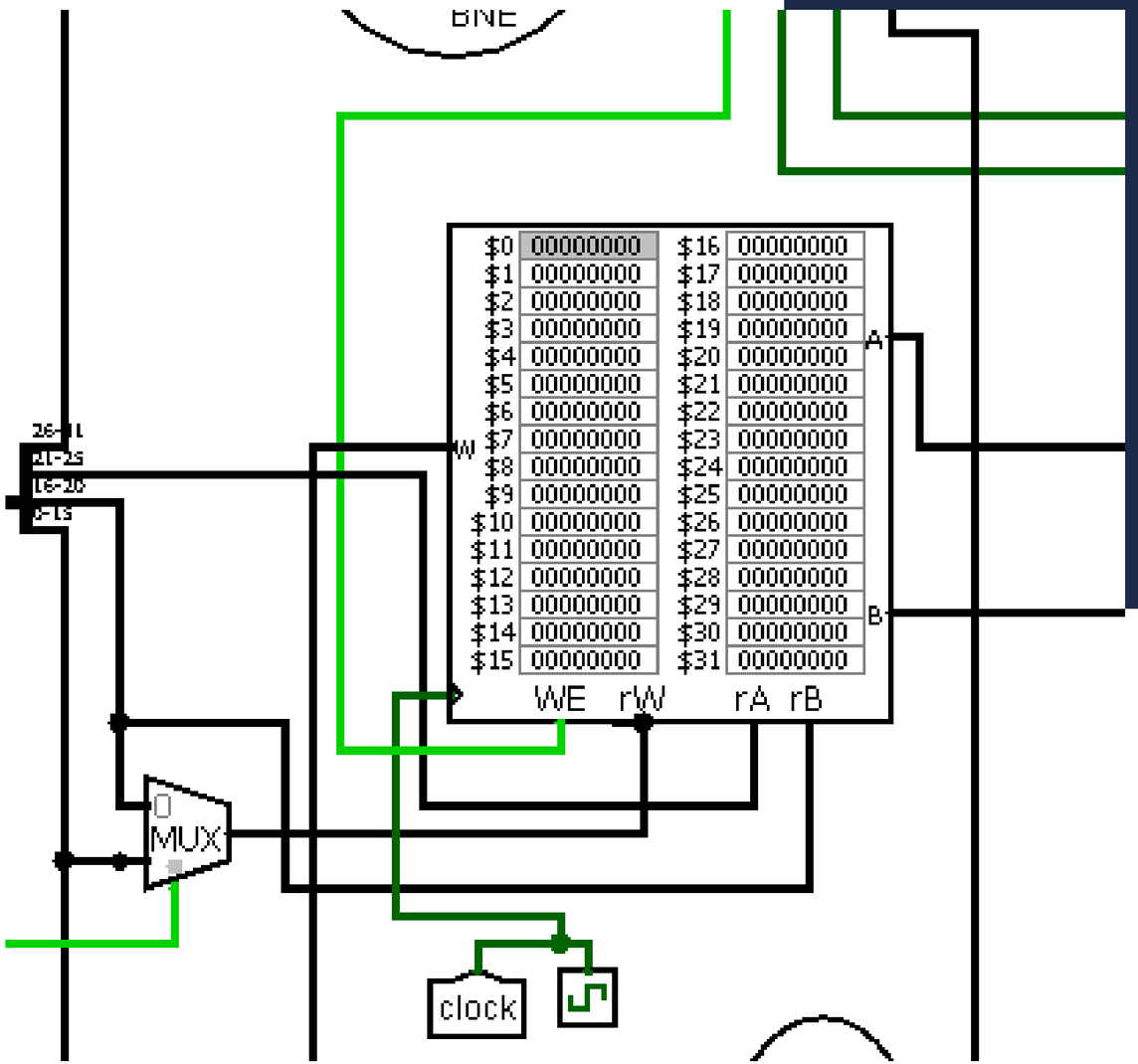
4. REGISTER FILE

01 **Purpose**: Stores general-purpose registers (R0–R31).

02 **Input**:

- Read register 1 (rs)
- Read register 2 (rt)
- Write register (rd or rt depending on RegDst)
- Write data

03 **Output**: Data from rs and rt.

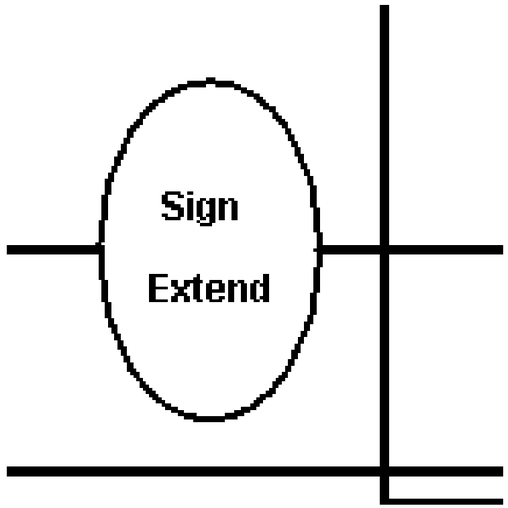


5. SIGN EXTEND

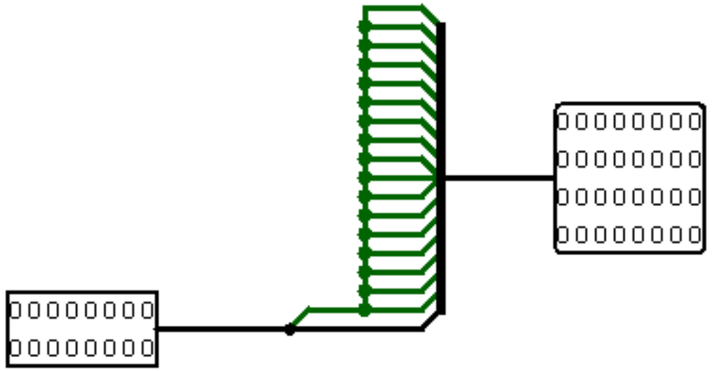
01 **Purpose**: Extends 16-bit immediate values to 32-bit.

02 **Why needed?** For I-type instructions like addi, beq, bne, etc.

03 **Effect**: Maintains sign (positive/negative).



Sign Extend



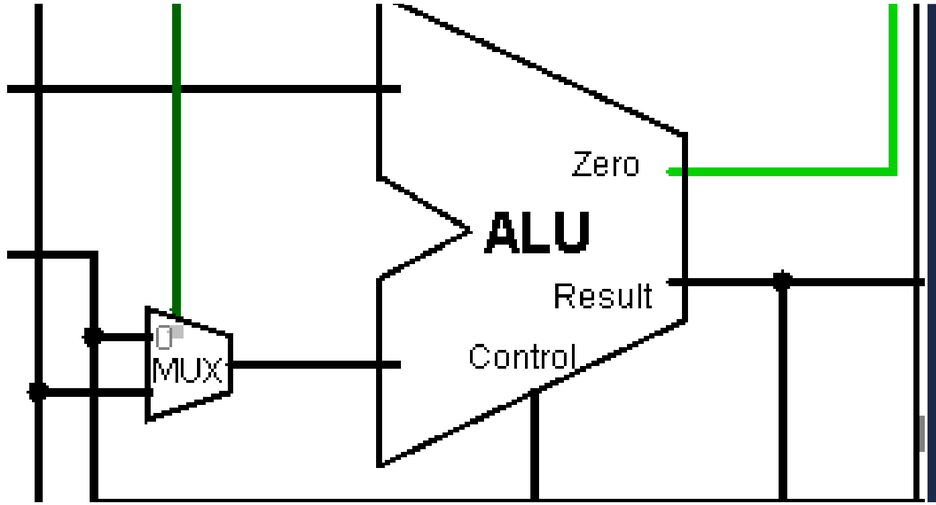
6. ALU (ARITHMETIC LOGIC UNIT)

01 **Purpose**: Performs arithmetic and logic operations.

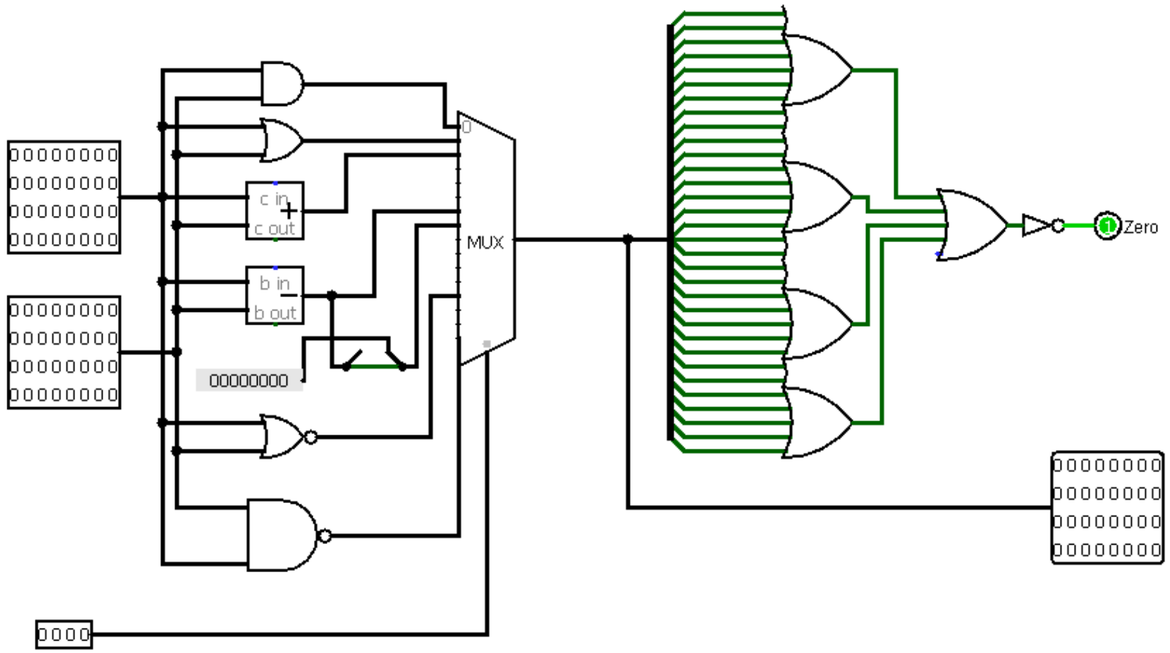
02 **Input**:

- Operand 1: rs
- Operand 2: rt or sign-extended immediate (selected via ALUSrc)
- ALU control signals

03 **Output**: Result and Zero flag (used by beq and bne).



ALU



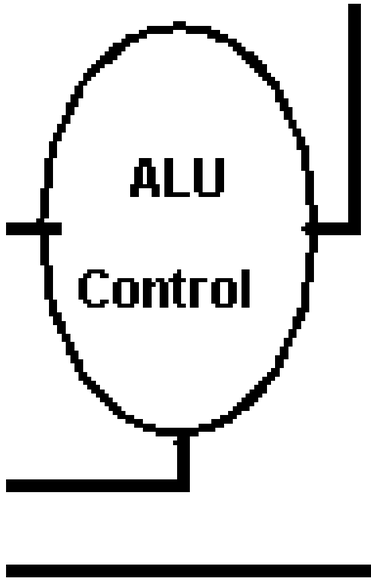
7. ALU CONTROL

01 **Purpose**: Decodes the funct field (R-type) or uses ALUOp to determine exact operation.

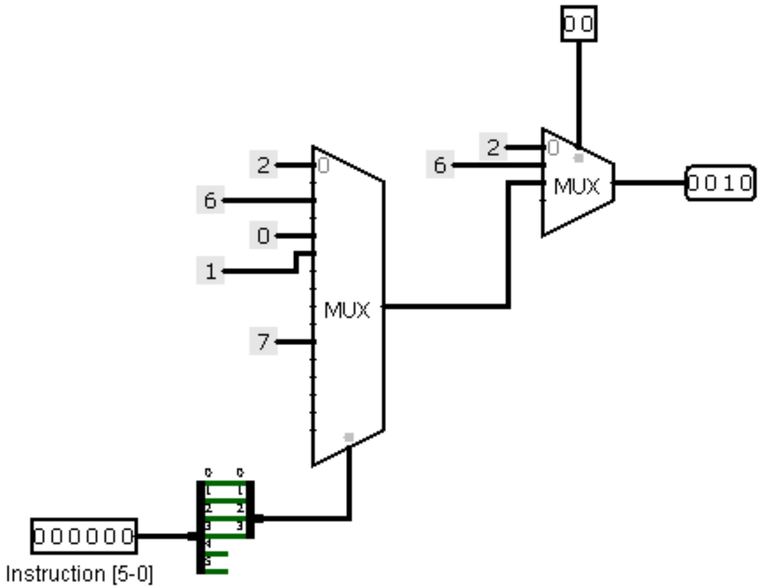
02 **Input**:

- ALUOp (from main control)
- Function field (funct) of R-type instructions

03 **Output**: ALU operation code (e.g., ADD, SUB, AND, etc.)



ALU Control



8. DATA MEMORYT

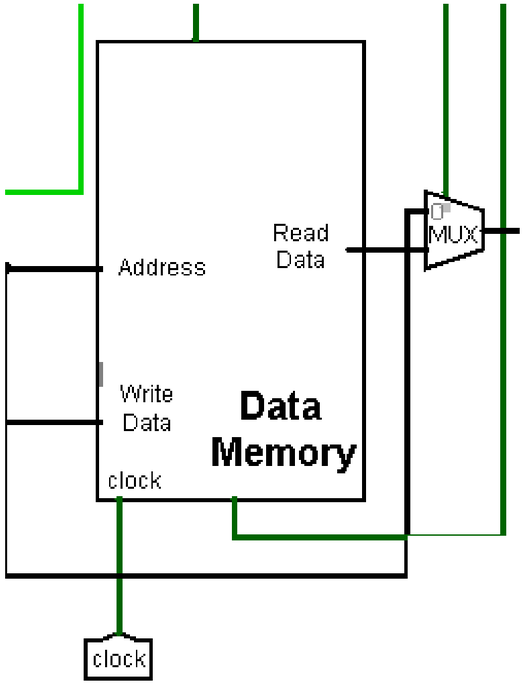
01 **Purpose**: Read/write memory for lw and sw.

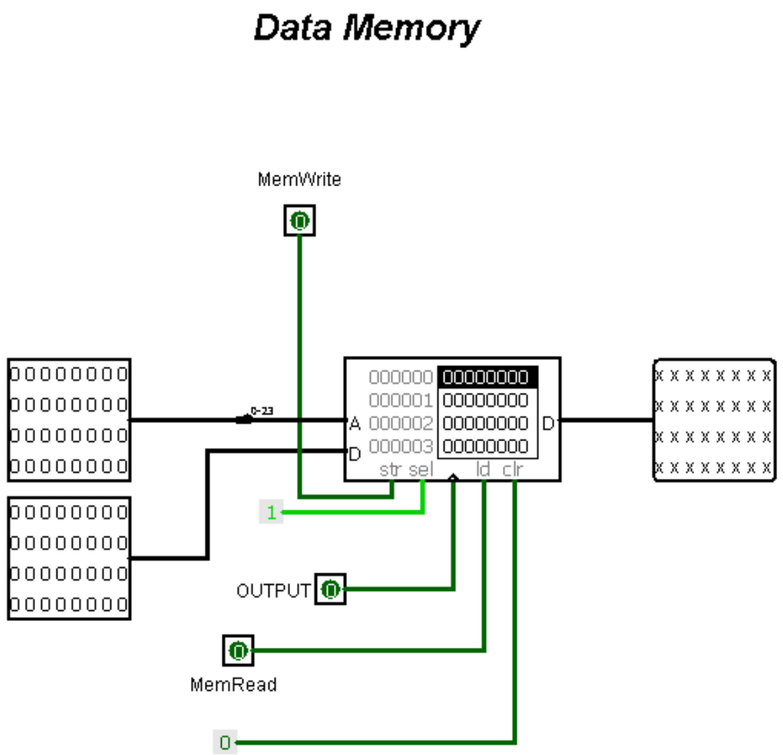
02 **Input**:

- Address (from ALU result)
- Write data (from register)
- MemRead / MemWrite control signals

ion [31:26] (opcode).

03 **Output**: Data read (for lw)



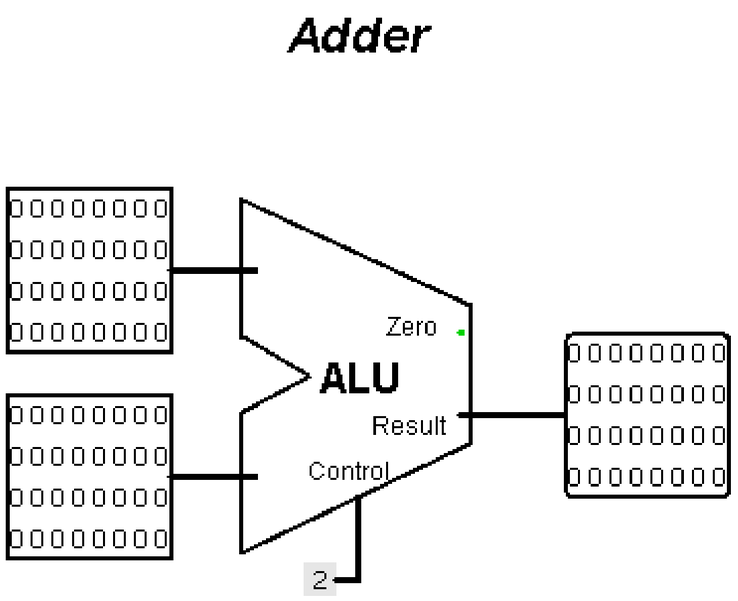


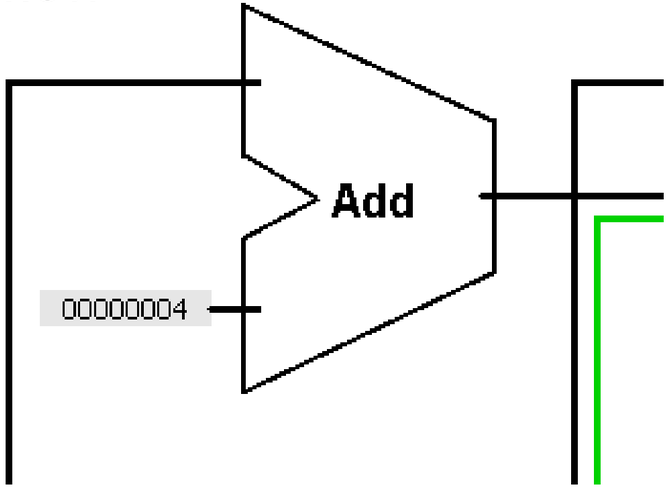
9. ADDERS

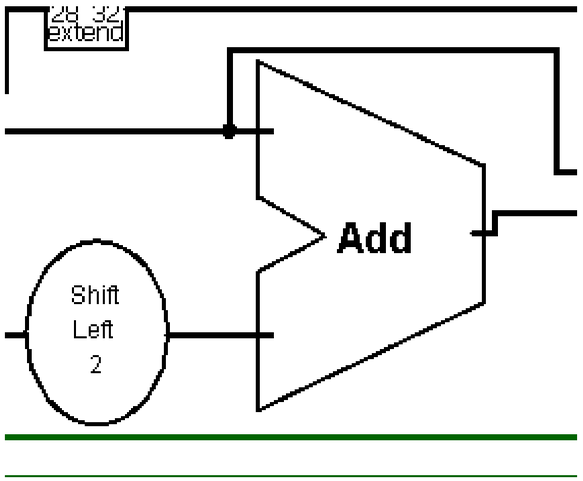
01 **Two Adders**

- First Adder: PC + 4 (next sequential instruction)
- Second Adder: PC + 4 + (offset for branch)

02 **Used in**: Normal instruction flow and branching.







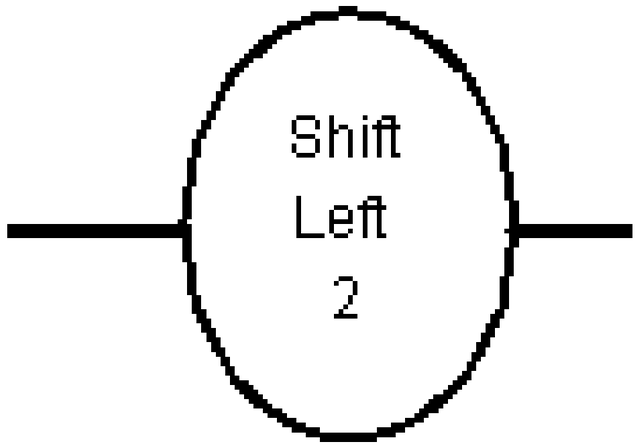
10. SHIFT LEFT 2

01 **Purpose**: Shifts 16-bit offset left by 2 bits to align with 4-byte word boundaries (used for branch).

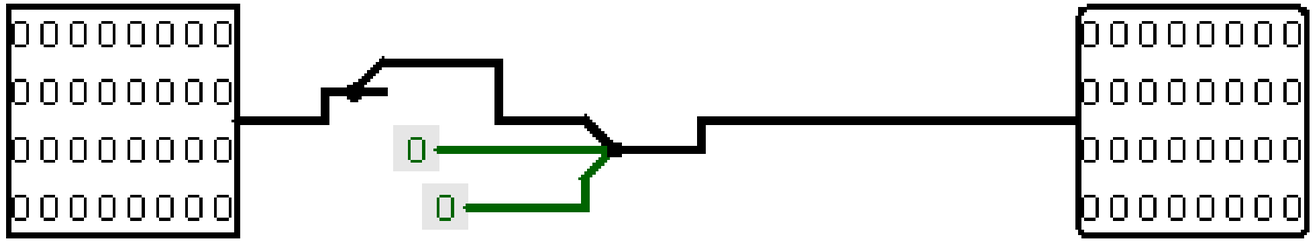
.

02 **Input**: Sign-extended immediate.

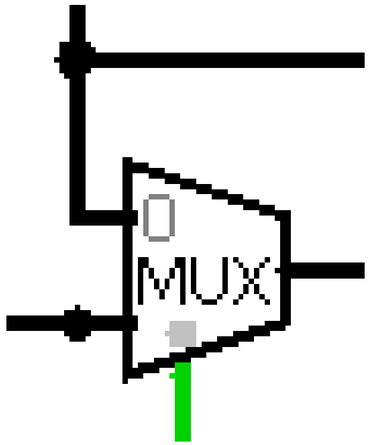
03 **Output**: Branch address offset.



Shift Left 2



11. MUXES (MULTIPLEXERS)



01 **MUX 1 (ALUSrc)**: Chooses between rt and immediate value.

02 **MUX 2 (RegDst)**: Chooses between rt and rd as the destination register.

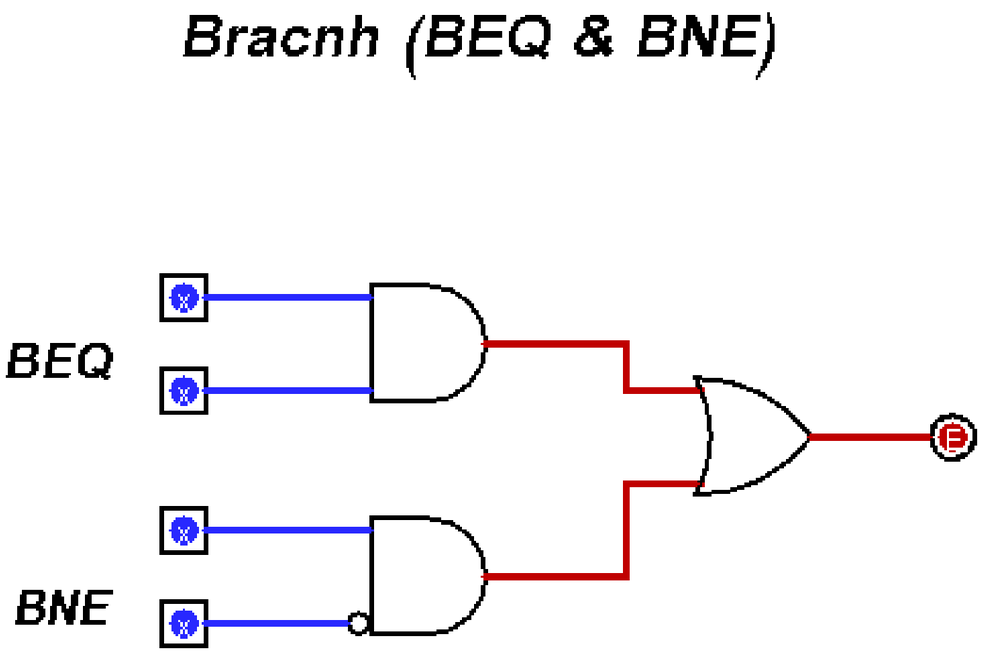
03 **MUX 3 (MemToReg)**: Chooses between ALU result and memory output for register write-back.

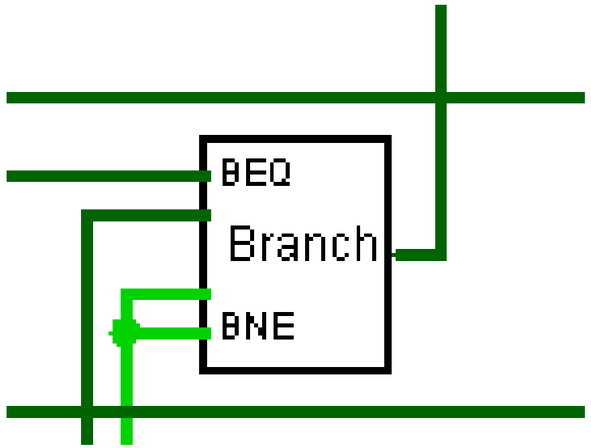
04 **MUX 4 (PCSrc)**: Chooses between sequential PC and branch target address.

12. BEQ & BNE LOGIC

01 **Purpose**: Controls branching.

02 **Input**: Zero signal from ALU and control bits Branch or BNE.





03 **Behavior:**

- For BEQ: branch if Zero = 1
- For BNE: branch if Zero = 0

04 **Output**: Controls PCSrc MUX for updating PC.

LET'S GO

Step-by-step in full detail for each instruction your 32-bit CPU supports. This will help you explain exactly how each **instruction works**, including **instruction format, bit-level encoding, CPU data path traversal,** and **control signal behavior.**

1. ADD — ADD REGISTERS

Datapath Steps:

1- **Instruction Fetch**: PC fetches instruction from memory.

2- **Instruction Decode**: Control unit identifies ADD. memory.

3- **Register Read**: Read R2 and R3 from register file.

4- **ALU Operation**: ALU adds R2 and R3.

5-**Write Back**: Result is written to R1.

Syntax:

- **ADD rd, rs, rt**

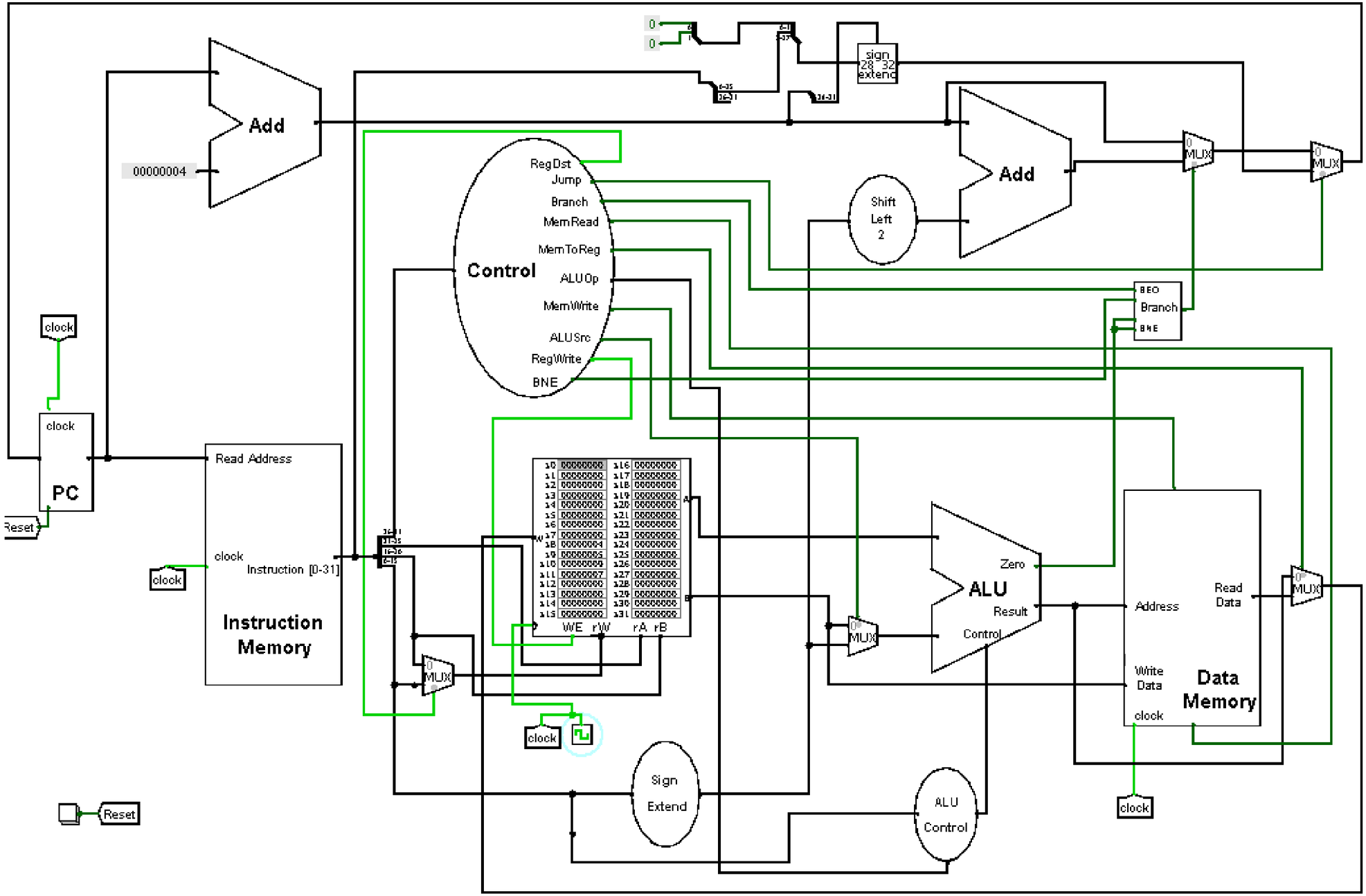
🔷 Format: R-type (Register type)

Field	Size	Description
opcode	6	Operation code (e.g., 000000 for R-type)
rs	5	First source register
rt	5	Second source register
rd	5	Destination register
shamt	5	Shift amount (0 for ADD)
funct	6	Function code (e.g., 100000 for ADD)

🔷 Control Signals:

Signal	Value	Purpose
RegDst	1	Use `rd` for destination
ALUSrc	0	Use register, not immediate
MemtoReg	0	Result comes from ALU
RegWrite	1	Enable writing to `rd`
ALUOp	10	ALU control interprets `funct`
MemRead	0	No memory read
MemWrite	0	No memory write
Branch	0	Not a branch instruction
Jump	0	Not a jump

Single Cycle Processer



2. SUB — SUBTRACT REGISTERS

Datapath Steps:

Same as ADD, but ALU performs subtraction.

Format: R-type:

Same layout as ADD.
| Funct for SUB = 100010 |

Control Signals::

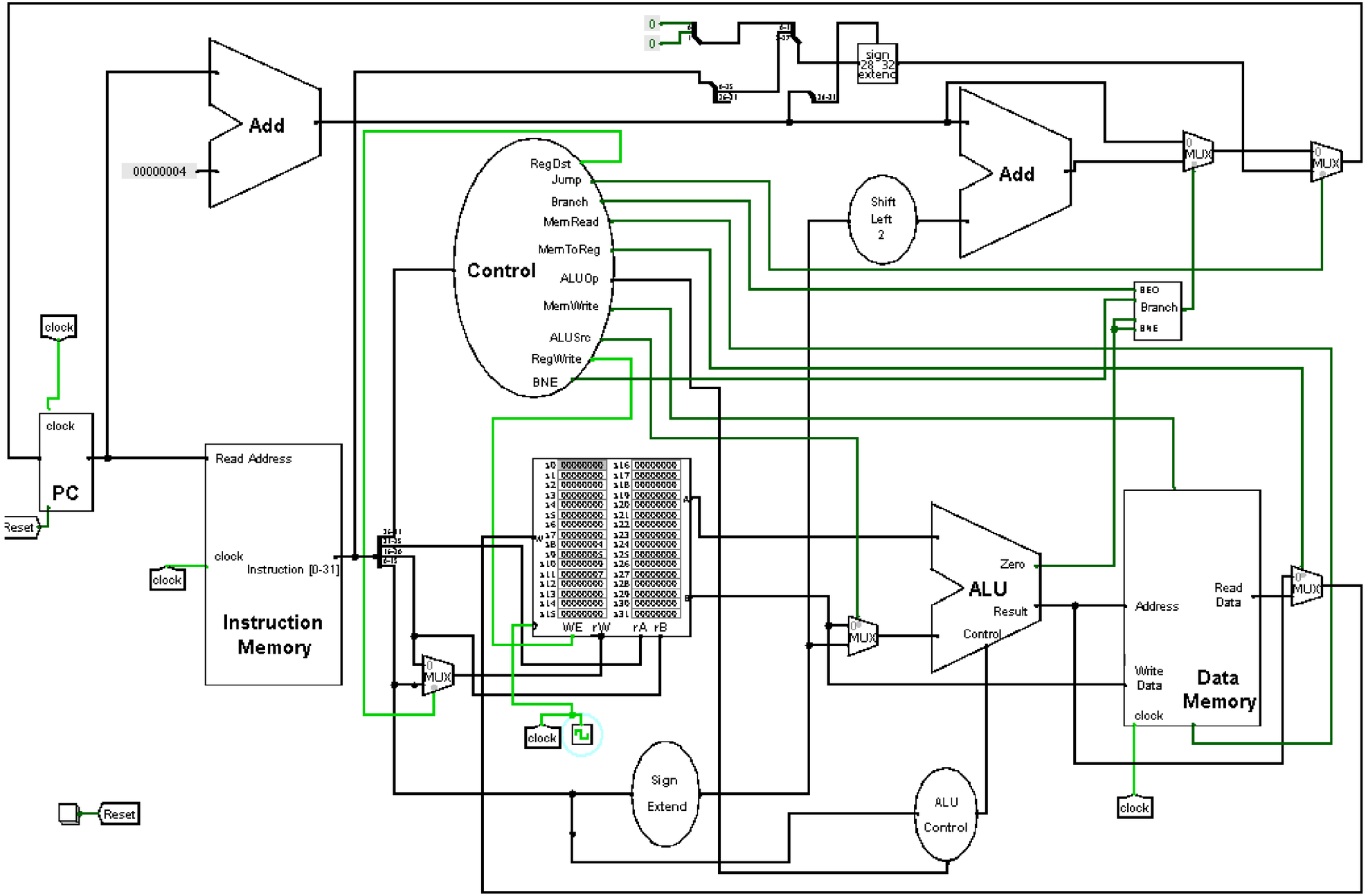
Same as ADD, except:

- ALUOp still = 10, but funct = 100010 maps to subtraction in ALU control.

Syntax:

- **SUB rd, rs, rt**

Single Cycle Processer



3. BEQ — BRANCH IF EQUAL

Datapath Steps:

1- **Fetch & Decode** instruction.

2- **Read Registers** R4 and R5.

3-**ALU Comparison:** R4 - R5

4-**Check Zero output:**

- If result is zero → branch is taken.

5-**Update PC** with offset (shifted left 2 bits for word addressing).

Syntax:

- **BEQ rs, rt, offset**

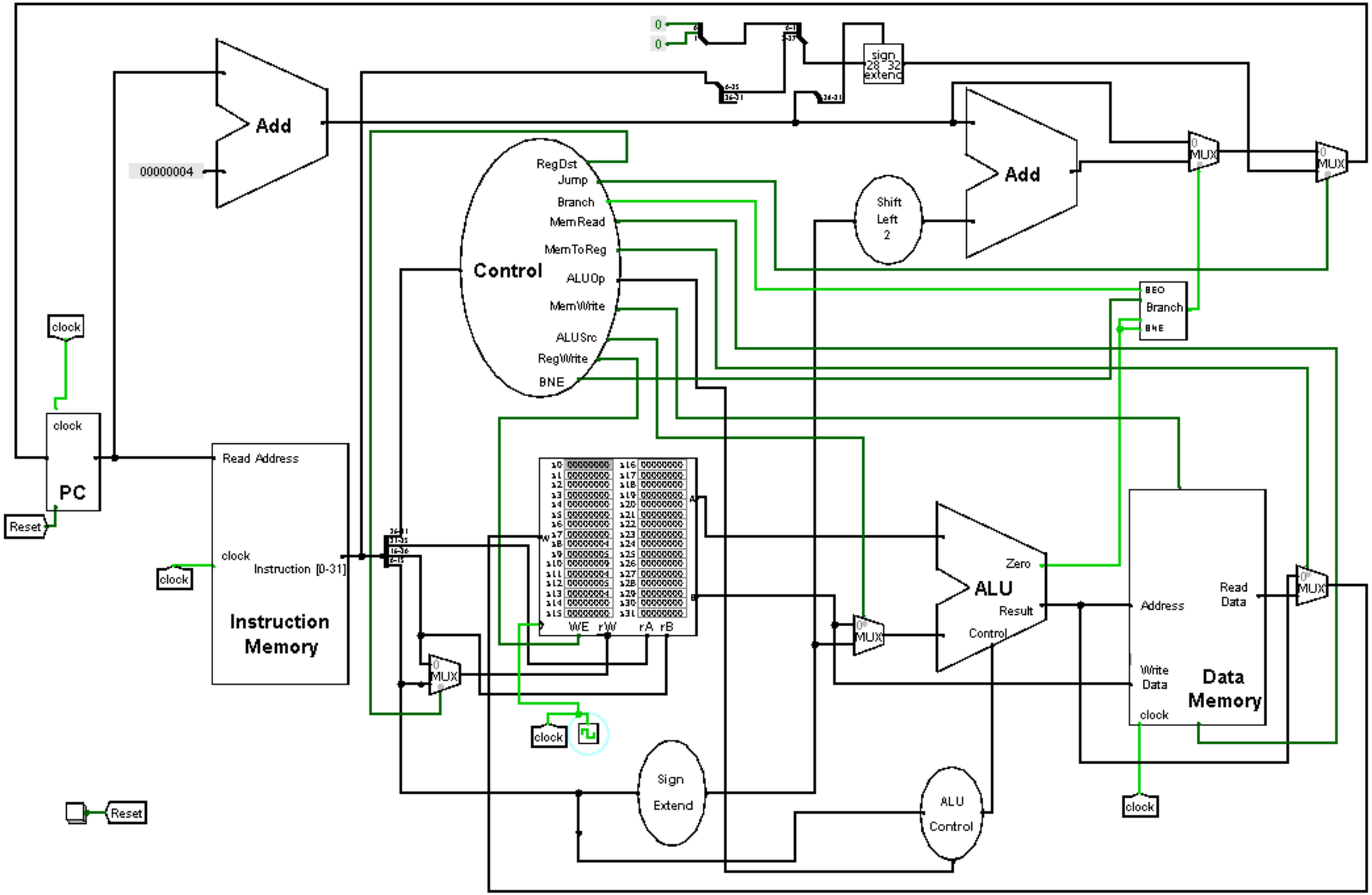
🔷 Format: I-type (Immediate)

Field	Size	Description
opcode	6	e.g., `000100` for BEQ
rs	5	First source register
rt	5	Second source register
offset	16	Signed offset for PC update

🔷 Control Signals:

Signal	Value	Purpose
ALUSrc	0	Use second register
ALUOp	01	Tells ALU to compare
Branch	1	Enable branch logic
PCSrc	Depends on ALU Zero output	

Single Cycle Processer



4. BNE — BRANCH IF NOT EQUAL

Datapath Steps:

Same as BEQ, except:

- Branch taken if ALU output is not zero.

Format: I-type

ame as BEQ, but opcode is different (000101 for BNE).

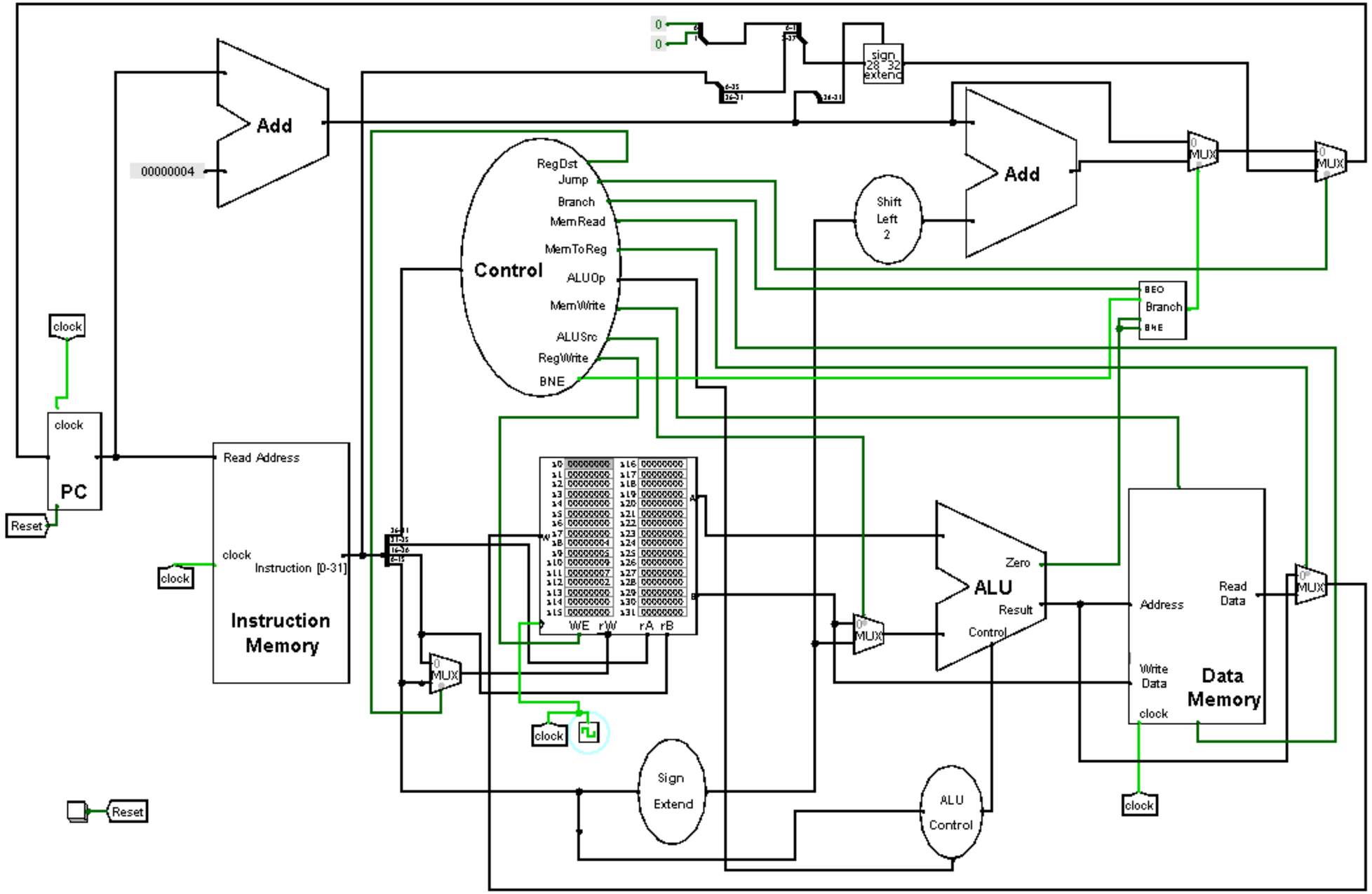
Control Signals::

Same as BEQ, but logic checks for Zero == 0 to take the branch.

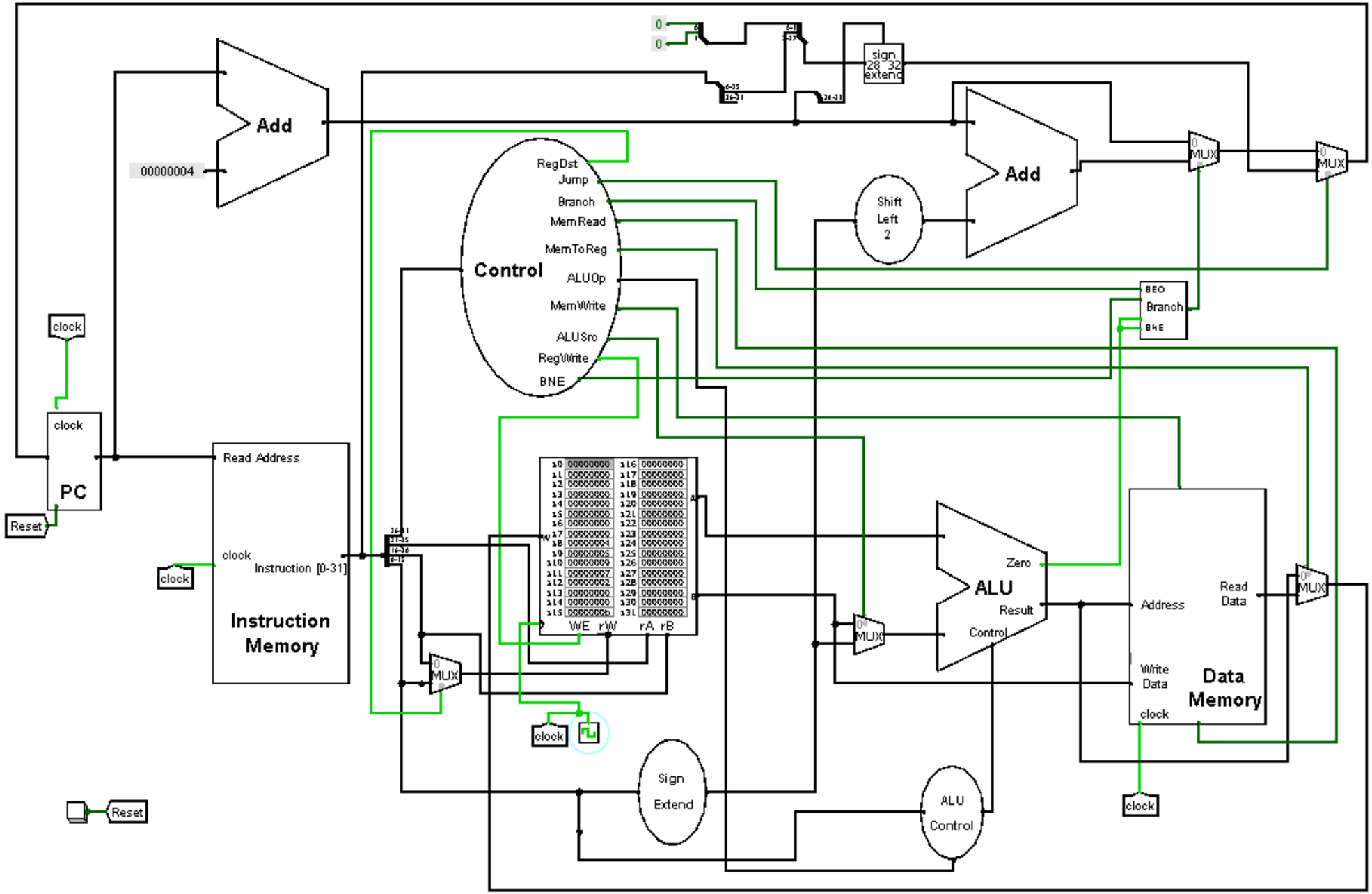
Syntax:

- **BNE rs, rt, offset**

Single Cycle Processer



Single Cycle Processer



5. JUMP — UNCONDITIONAL JUMP

Datapath Steps:

1- **Fetch & Decode** instruction.

2-**Control unit** identifies jump.

3-**New PC = (PC[31:28] << 28) | (address << 2)**

Syntax:

- **JUMP address**

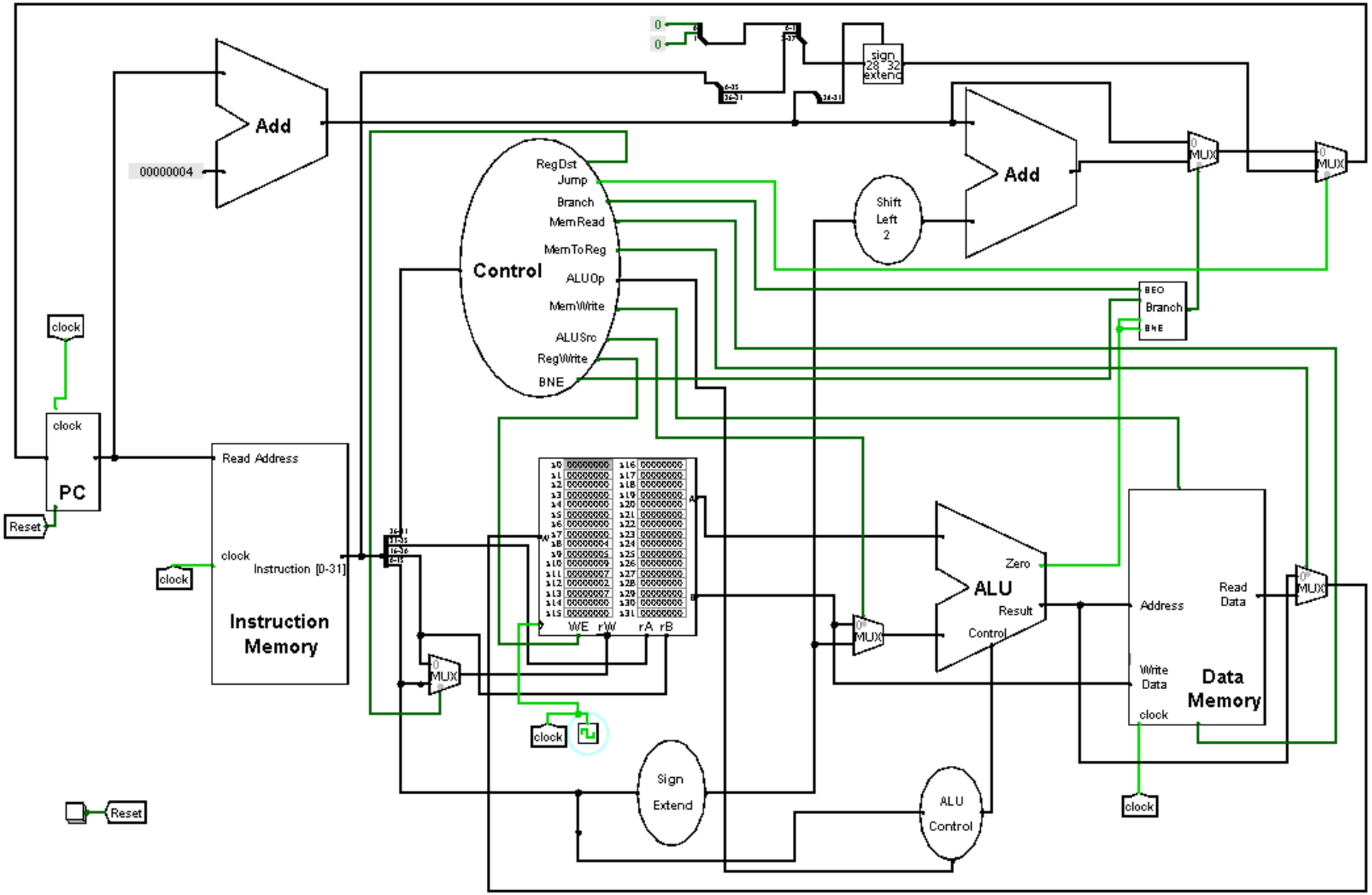
🔷 Format: J-type (Jump)

Field	Size	Description
opcode	6	e.g., `000010` for JUMP
address	26	Jump target address (word-aligned)

🔷 Control Signals:

Signal	Value	Purpose
Jump	1	Jump to target
PCSrc	1	Use jump address
ALUOp	x	Don't care
RegWrite	0	No register update

Single Cycle Processer



6. SLT — SET ON LESS THAN

Datapath Steps:

1- **Instruction Memory**: Reads the slt instruction.

2- **Register File**: Reads values from rs and rt.

3- **ALU Control:**

- ALUOp = 10 (for R-type).
- funct = 101010 → ALU Control outputs the SLT operation

4- **ALU** : Compares rs and rt. If rs < rt, result = 1, else result = 0

5-**Write Back**: Result is written to rd register via the Register File.

Syntax:

- **SLT rd, rs, rt**

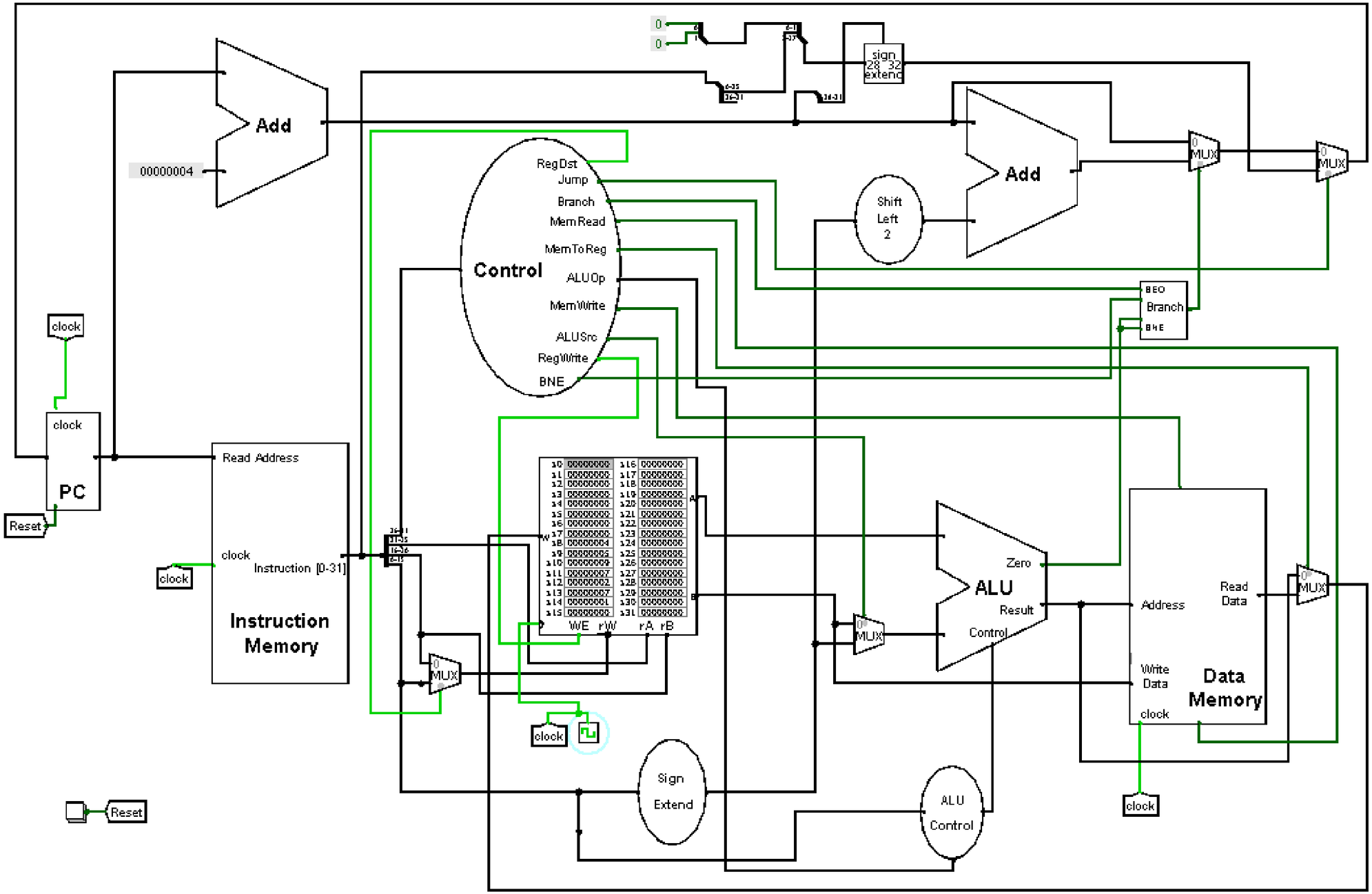
◆ Instruction Format (R-type)

Field	Bits	Description
opcode	6	`000000` (R-type)
rs	5	First source register
rt	5	Second source register
rd	5	Destination register
shamt	5	Shift amount (0 for SLT)
funct	6	`101010` for SLT

◆ Control Signals

Signal	Value	Comment
RegDst	1	Choose `rd` as write register
ALUSrc	0	Second ALU operand comes from register
MemtoReg	0	Write ALU result, not memory data
RegWrite	1	Enable register write
MemRead	0	No data memory read
MemWrite	0	No data memory write
Branch	0	Not a branch
Jump	0	Not a jump
ALUOp	10	"R-type" operation → further decode by funct

Single Cycle Processer



7. SW – STORE WORD

Datapath Steps:

1- **Instruction Fetch**:

2- Decode and read rs and rt

3- **ALU calculates address:** Address = R[rs] + offset

4-**Memory write**: store data from R[rt] to memory

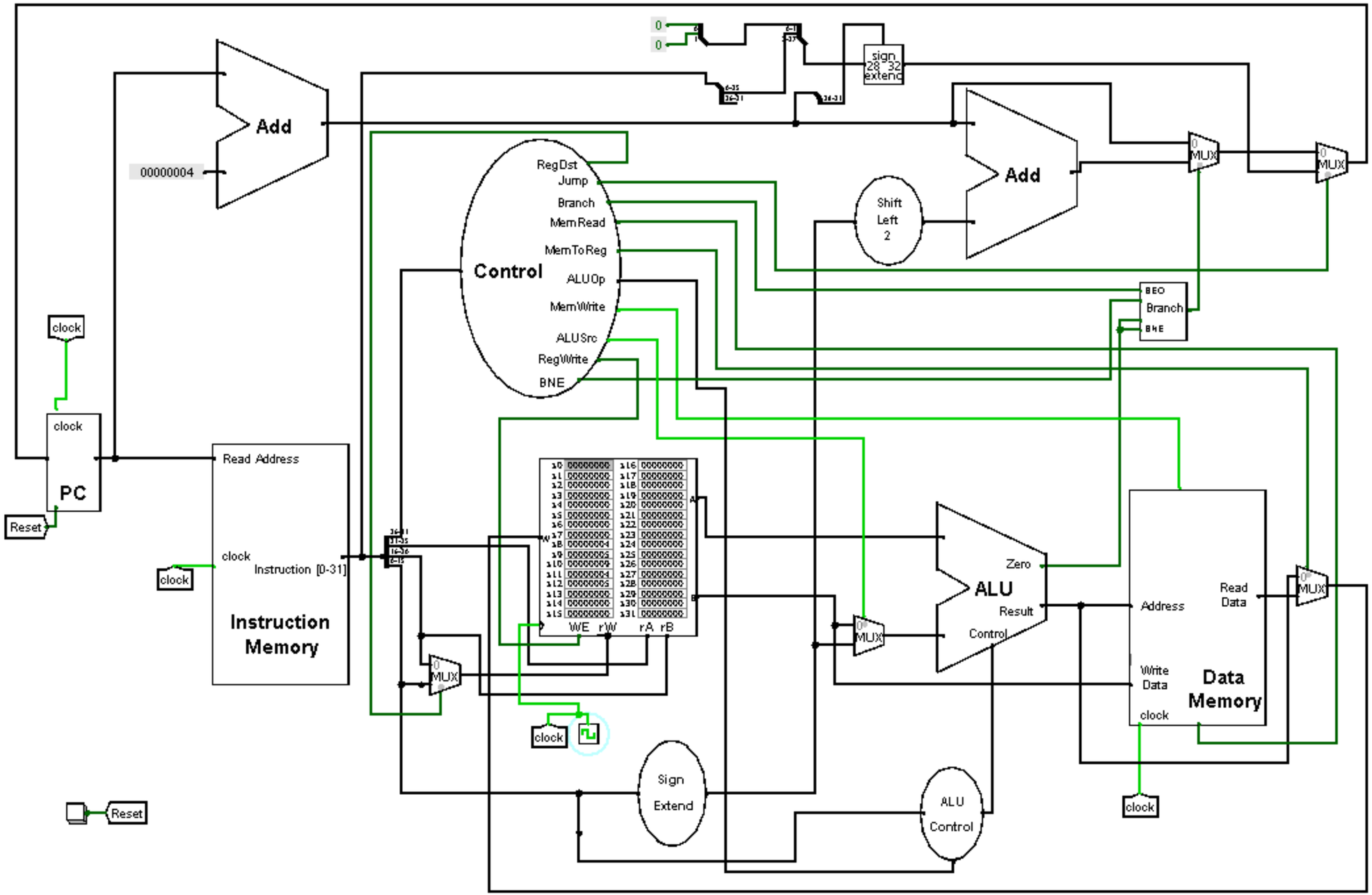
Syntax:

- **SW rt, offset(rs)**

◆ Control Signals for `SW`

Signal	Value	Comment
RegDst	X	Don't care (no register write)
ALUSrc	1	ALU gets second input from immediate
MemtoReg	X	Don't care
RegWrite	0	No write to register
MemRead	0	Not reading from memory
MemWrite	1	Write to memory
Branch	0	Not a branch
ALUOp	00	ALU performs addition

Single Cycle Processer



8. LW – LOAD WORD

Datapath Steps:

1- **Instruction Fetch**:

2- Decode and read rs

3- **ALU calculates address:** Address = R[rs] + offset

4-**Memory read** at Address

5-**Write result** to register rt

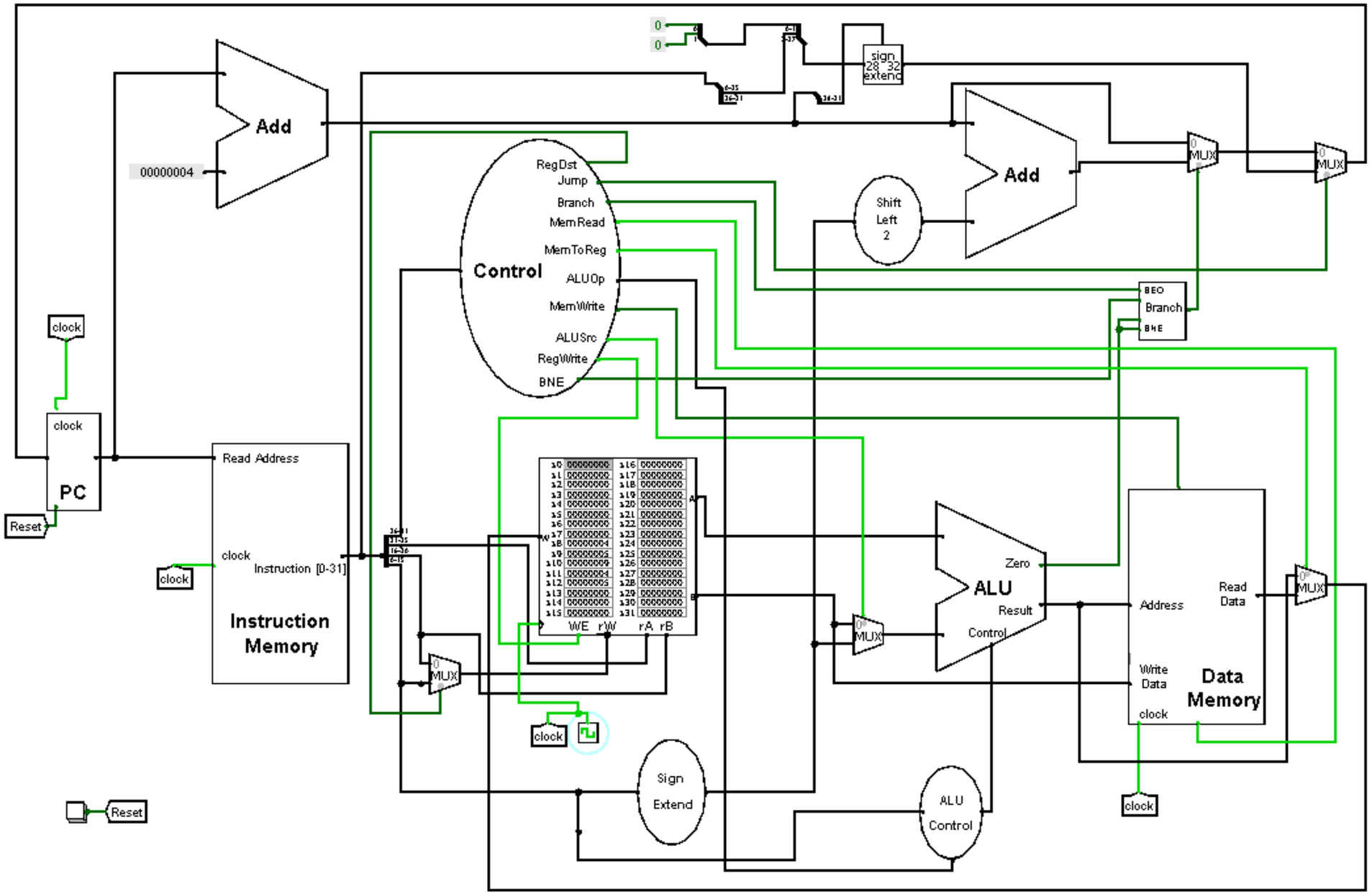
Syntax:

- **LW rt, offset(rs)**

◆ Control Signals for `LW`

Signal	Value	Comment
RegDst	0	Write to `rt`
ALUSrc	1	ALU gets second input from immediate
MemtoReg	1	Write data from memory
RegWrite	1	Write to register
MemRead	1	Read from memory
MemWrite	0	Not writing to memory
Branch	0	Not a branch
ALUOp	00	Just add offset

Single Cycle Processer



🔷 Instruction Format (I-type)

Field	Bits	Description
opcode	6	`100011` for LW, `101011` for SW
rs	5	Base register
rt	5	Target/source register
imm	16	Offset (signed)

9. AND — BITWISE AND

Control Signals:

Same as ADD (R-type), but
funct = 100100 → ALU performs AND.

Format: R-type

Field	Bits	Description
opcode	6	`000000` (R-type)
rs	5	First source register
rt	5	Second source register
rd	5	Destination register
shamt	5	Shift amount (0 here)
funct	6	`100100` for AND

Syntax:

- **AND rd, rs, rt**

10. NAND — BITWISE NAND

🔹Format: R-type

You need a unique funct code, e.g., 101100

📌 This is not in standard MIPS, but many custom CPUs support NAND logic.

🔹 Control Signals::

- ALUOp = 10 → ALU uses funct field
- ALU Control decodes funct = 101100 to NAND

🔹 Syntax:

- **NAND rd, rs, rt**

11. OR — BITWISE OR

- **Format: R-type**

  | Funct = 100101 |

- **Control Signals::**

  Same as AND, but funct = 100101.

- **Syntax:**

  - **OR rd, rs, rt**

12. NOR — BITWISE NOR

- **Format: R-type**

  | Funct = 100111 |

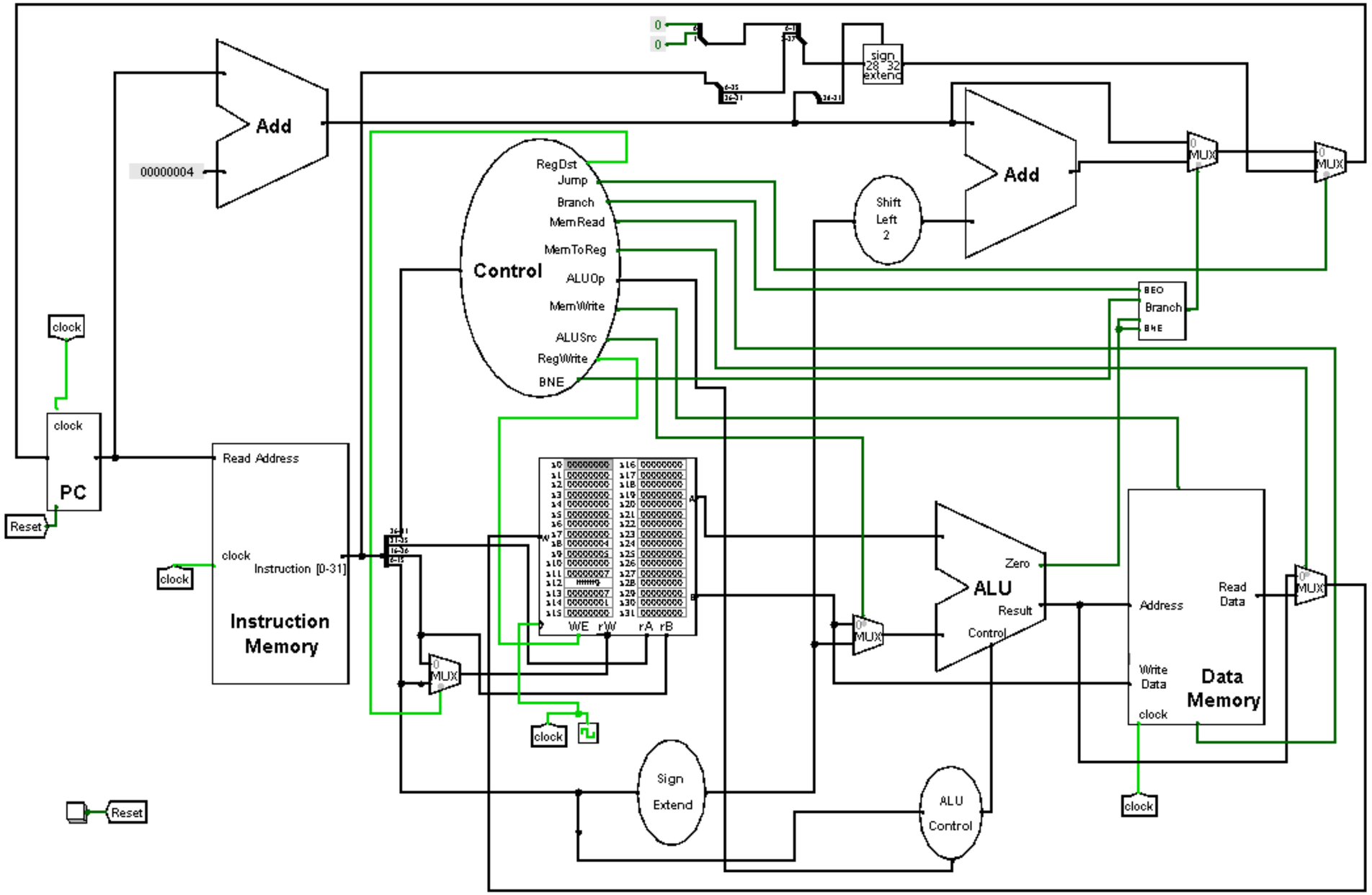
- **Syntax:**

  - **NOR rd, rs, rt**

DATAPATH NOTES FOR ALL LOGIC

1- **Instruction Fetch** → PC gets instruction.
2- **Decode** → Control unit reads opcode 000000.
3- **Register Read** → Get rs, rt values.
4- **ALU Logic** → ALU performs logic operation using funct.
5- **Write Back** → Result saved to rd.

Single Cycle Processer



THANK YOU!

For your attention